

Universal AI API Connector & API Hub

Production-Grade Multi-Provider AI Gateway & Management Platform

Status: Verified Production Live

Architecture: Full-Stack TypeScript

Persistence: SQLite WAL

Test Coverage: 24/24 Tests Passing

Live Deployment URL: <https://apiconnector-ln3y.onrender.com/>
GitHub Repository: <https://github.com/saurabhhumane125/APIconnector>

1. Executive Summary & System Overview

The **Universal AI API Hub** solves the problem of chaotic, ad-hoc AI integration in modern engineering teams. Instead of hardcoding vendor-specific client SDKs (OpenAI, Gemini, Groq, Anthropic), parameter formatting, and bespoke prompt engineering inside downstream applications, the Hub allows teams to configure structured **AI Connectors** through a graphical dashboard or REST API.

Each configured connector automatically generates:

- Dedicated Execution Endpoints:** `POST /api/v1/run/:slug` and `POST /api/v1/connectors/:id/execute`.
- Dynamic Input Validation:** Strict validation of parameters (text, numbers, booleans, enums, JSON, and multimodal Base64 images).
- Prompt & Schema Binding:** Automated injection of dynamic user inputs into custom system instructions with mandatory JSON output schemas.
- Normalized Response Envelope:** Standardized `{ success, data, error, meta }` payload format with millisecond latency, token usage, and micro-cost calculations.
- Persistent Auditable Telemetry:** SQLite database with Write-Ahead Logging (WAL) storing complete request and response payloads, status codes, and client traces.

2. System Architecture

The platform follows a layered, decoupled service architecture:

Layer	Technologies	Key Responsibilities
Web UI	React 19, TypeScript, Vite, Custom Design System	Admin dashboard, connector creator/editor, interactive API tester, credential manager, telemetry log inspector.
API Gateway	Express 4, TypeScript, CORS, JSON Stream	Dynamic route dispatcher, request timing, correlation ID generation (<code>X-Request-Id</code>), body size limiting (25MB).
Execution Engine	TypeScript Domain Services	Schema validation, prompt interpolation, cryptographic API key verification, multi-tier failover orchestrator.
Provider Adapters	Multi-Provider Adapter Pattern	Gemini (v1beta REST with AQ Authorization headers), Groq LPU, OpenAI, and Anthropic Claude adapters.
Persistence	SQLite (better-sqlite3), WAL Mode	Connectors, parameters, API keys, provider settings, schema migrations, and immutable execution logs.

3. High-Availability & Fault Tolerance Engine

To guarantee 99.99% operational reliability and prevent upstream provider outages from impacting client applications, the hub implements an enterprise-grade 4-tier resilience engine:

1. **Multi-Key Pooling & Load Balancing:** Administrators can supply multiple API keys per provider (comma or newline separated). The store balances traffic across keys and instantly rotates if any key encounters capacity or quota limits.
2. **Intra-Provider Sibling Model Failover:** When Google Gemini returns HTTP 503 (high demand) or 429 (rate limits) on `gemini-3.8-flash`, the adapter dynamically fails over across `gemini-3.5-flash`, `gemini-2.5-flash`, and `gemini-2.0-flash`.
3. **Transient Spike Jittered Backoff:** Automatic 600ms exponential retry for momentary upstream cluster load rebalancing.
4. **Cross-Provider Automatic Redundancy:** If a provider suffers a catastrophic outage, the execution engine dynamically routes text tasks to Groq or OpenAI, and multimodal vision tasks to OpenAI (`gpt-4o-mini`), returning HTTP 200 with complete failover audit metadata.

4. Core Functional Modules & Verification Evidence

4.1 Automated Test Suite Results

The codebase includes an exhaustive test suite executed via **Vitest** covering unit contracts, database migrations, dynamic validation, provider adapters, and end-to-end execution:

```
✓ tests/unit/providerLayer.test.ts (6 tests)
✓ tests/unit/connectorCore.test.ts (6 tests)
✓ tests/integration/apiHubExecution.test.ts (6 tests)
✓ tests/unit/foundation.test.ts (3 tests)
✓ tests/e2e/apiHub.test.ts (3 tests)
  ✓ Phase 11, 12, 13 - End-to-End System & Reliability Verification >
    complete lifecycle: create connector -> inspect docs -> generate API key -> execute -> check telemetry

Test Files: 5 passed (5)
Tests:      24 passed (24)
Duration:   1.86s
Result:     100% Passing (Exit Code 0)
```

4.2 Demonstration Connectors (Pre-Seeded)

Slug	Provider & Model	Input Types	Verified Output Structure
support-ticket-triage	Groq / llama-3.3-70b-versatile	Text (ticket), Enum (tier), Number (urgency)	{ category, priority, sentiment, summary, suggested_action, escalation_required }
receipt-ocr-extractor	Gemini / gemini-3.8-flash	Image (Base64/URL), Text (currency), Boolean (notes)	{ merchant_name, date, currency, total_amount, tax_amount, line_items: [...] }

4.3 Sample Validated Execution Payload

```
{
  "success": true,
  "data": {
    "category": "billing",
    "priority": "high",
    "sentiment": "frustrated",
    "summary": "Customer charged twice for subscription renewal and requested an immediate refund.",
    "suggested_action": "Verify Stripe invoice charges, initiate refund for duplicate transaction, and no",
    "escalation_required": false
  },
  "error": null,
  "meta": {
    "requestId": "req_8x29a1b_mumb102",
    "connectorId": "conn_support_triage_001",
    "connectorSlug": "support-ticket-triage",
    "provider": "groq",
    "model": "llama-3.3-70b-versatile",
    "durationMs": 384,
    "tokens": {
```

```
"prompt": 142,  
"completion": 68,  
"total": 210  
},  
"estimatedCost": 0.000042  
}  
}
```

5. REST API Specifications

Method & Path	Purpose	Auth Required
GET /api/health	Gateway and database connectivity check with configured provider flags	No
GET /api/connectors	List all configured AI connectors	No
POST /api/connectors	Create a new dynamic AI connector definition	Admin
POST /api/v1/run/:slug	Execute an AI connector by URL slug	Configurable (Bearer / Header / None)
POST /api/v1/connectors/:id/execute	Execute an AI connector by unique ID	Configurable
GET /api/connectors/:id/docs	Retrieve auto-generated OpenAPI documentation & cURL samples	No
GET /api/metrics/summary	Cumulative request counts, error rates, avg latency, and costs	No
GET /api/metrics/logs	Paginated audit log records with query filters	No
POST /api/keys	Generate a new scoped, hashed API client key	No
DELETE /api/keys/:id	Immediately revoke an active API key	No
POST /api/providers/:id/key	Configure, update, or pool API credentials directly in-app	No

6. Known Limitations & Future Roadmap

- **Streaming Output (SSE):** The current release focuses on strict structured JSON validation; Server-Sent Events (SSE) streaming for free-form conversational tokens is scheduled for v1.1.
- **Database Storage Tier:** Built with local SQLite in WAL mode for microsecond response times and zero external database costs; enterprise clusters with multi-region write replication can swap to Postgres via the connection interface.
- **Semantic Caching:** Exact JSON input hashing is supported; embedding-based semantic vector similarity caching is planned for v1.2.